MIPT & YSDA · AUTUMN 2026

Deep Generative Models

Lecture 9

Roman Isachenko

Moscow Institute of Physics and Technology
Yandex School of Data Analysis

Recap of Previous Lecture

Reparametrization of DDPM

Let $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod_{s=1}^{t} \alpha_s = \prod_{s=1}^{t} (1 - \beta_s)$.

Forward Noising

$$\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$$

Recover Clean Data

$$\mathbf{x}_0 = \frac{\mathbf{x}_t - \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}}{\sqrt{\bar{\alpha}_t}}$$

Why reparametrize? Predict the Gaussian noise we add ourselves during training; compute the reverse mean analytically from this prediction.

Target Mean

$$\tilde{\boldsymbol{\mu}}_t(\mathbf{x}_t, \mathbf{x}_0) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}$$

Model Mean

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{1}{\sqrt{\alpha_t}} \cdot \mathbf{x}_t - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \cdot \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)$$

Reverse-mean, clean-data and noise predictions are convertible; unweighted losses assign different weights to noise levels.

Ho J. et al. Denoising Diffusion Probabilistic Models, 2020
Salimans T., Ho J. Progressive Distillation for Fast Sampling of Diffusion Models, 2022

Recap of Previous Lecture

Training

1. Sample $\mathbf{x}_0 \sim p_{\text{data}}(\mathbf{x})$, $t \sim \text{Uniform}\{1, \dots, T\}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.
2. Compute noisy image $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$.
3. Minimize $\mathcal{L}_{\text{simple}} = \|\boldsymbol{\epsilon} - \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t)\|^2$ by SGD.

Sampling (Ancestral)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$.
2. For $t = T, \dots, 2$, compute the mean and sample the reverse step:

$$\boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \frac{\mathbf{x}_t}{\sqrt{\alpha_t}} - \frac{1 - \alpha_t}{\sqrt{\alpha_t(1 - \bar{\alpha}_t)}} \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t),$$

$$\mathbf{x}_{t-1} = \boldsymbol{\mu}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \sqrt{\tilde{\beta}_t} \boldsymbol{\epsilon}_t, \qquad \boldsymbol{\epsilon}_t \sim \mathcal{N}(0, \mathbf{I}).$$

3. Return the decoder mean $\hat{\mathbf{x}}_0 = \boldsymbol{\mu}_{\boldsymbol{\theta},1}(\mathbf{x}_1)$.

Recap of Previous Lecture

DDPM vs NCSN: Summary

DSM: learn $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t)$ using conditional targets:

$$\ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{p_{\text{data}}(\mathbf{x}_0)} \mathbb{E}_{q(\mathbf{x}_t|\mathbf{x}_0)} \left\| \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) - \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{x}_0) \right\|_2^2.$$

Gaussian perturbations:

- DDPM: $\mathbf{x}_t = \sqrt{\bar{\alpha}_t} \cdot \mathbf{x}_0 + \sqrt{1 - \bar{\alpha}_t} \cdot \boldsymbol{\epsilon}$;
- NCSN: $\mathbf{x}_t = \mathbf{x}_0 + \sigma_t \cdot \boldsymbol{\epsilon}$.
- Use $\tilde{\mathbf{x}}_t = \mathbf{x}_t / \sqrt{\bar{\alpha}_t}$ and $\sigma_t^2 = (1 - \bar{\alpha}_t)/\bar{\alpha}_t$ to recover NCSN.

Scale target and model scores by $\sqrt{\bar{\alpha}_t}$ to obtain the rescaled DSM error:

$$\tilde{\ell}_{\boldsymbol{\theta}}(t) = \bar{\alpha}_t \ell_{\boldsymbol{\theta}}(t), \qquad \sigma_t^2 \tilde{\ell}_{\boldsymbol{\theta}}(t) = (1 - \bar{\alpha}_t) \ell_{\boldsymbol{\theta}}(t).$$

DDPM: ancestral transitions between noise levels.
NCSN: Langevin MCMC at each fixed noise level.

Vincent P. A Connection Between Score Matching and Denoising Autoencoders, 2010
Ho J., et al. Denoising Diffusion Probabilistic Models, 2020
Kingma D. et al. Variational Diffusion Models, 2021
Song Y. et al. Score-Based Generative Modeling through Stochastic Differential Equations, 2020

Recap of Previous Lecture

Loss Weighting and Time Sampling

DDPM replaces ELBO weights with a simple noise-prediction loss and generates through Gaussian reverse steps.

Fix the perturbation and its per-step loss $\ell_{\boldsymbol{\theta}}(t)$; let $w_t > 0$.

$$\mathcal{L} = \frac{1}{T} \sum_{t=1}^{T} w_t \ell_{\boldsymbol{\theta}}(t).$$

Sampling a noise level more often gives it more weight

$$\mathbb{E}_{t \sim r}[\ell_{\boldsymbol{\theta}}(t)] = \sum_{t=1}^{T} r_t \ell_{\boldsymbol{\theta}}(t) = \mathbb{E}_{t \sim \mathrm{Uniform}\{1,\ldots,T\}}[T r_t \, \ell_{\boldsymbol{\theta}}(t)] \,.$$

Conversely, choose $r_t = w_t / \sum_s w_s$ to absorb the weights, up to an overall loss scale.

To change time sampling while preserving a chosen objective, compensate:

$$\mathcal{L} = \mathbb{E}_{t \sim r} \left[\frac{w_t}{T r_t} \ell_{\boldsymbol{\theta}}(t) \right], \qquad r_t > 0.$$

The objective stays the same; gradient variance can change. These are **training** times.

Recap of Previous Lecture

Sampling (DDIM)

1. Sample $\mathbf{x}_T \sim \mathcal{N}(0, \mathbf{I})$; choose a grid $T = t_K > \cdots > t_0 = 0$.
2. For $k = K, \ldots, 1$, set $t = t_k$, $s = t_{k-1}$ and compute:

$$\hat{\boldsymbol{\epsilon}} = \boldsymbol{\epsilon}_{\boldsymbol{\theta},t}(\mathbf{x}_t) \qquad \text{predict noise,}$$

$$\hat{\mathbf{x}}_0 = \frac{\mathbf{x}_t - \sqrt{1-\bar{\alpha}_t}\,\hat{\boldsymbol{\epsilon}}}{\sqrt{\bar{\alpha}_t}} \qquad \text{estimate clean data,}$$

$$\mathbf{x}_s = \sqrt{\bar{\alpha}_s}\,\hat{\mathbf{x}}_0 + \sqrt{1-\bar{\alpha}_s}\,\hat{\boldsymbol{\epsilon}} \qquad \text{update state.}$$

3. Return $\mathbf{x}_0$.

Same Gaussian marginals $q(\mathbf{x}_t \mid \mathbf{x}_0)$; a different joint $q(\mathbf{x}_{1:T} \mid \mathbf{x}_0)$.

Deterministic steps: randomness comes from $\mathbf{x}_T$; recompute both estimates at every step.

Fewer steps: choose $K \ll T$ on the original schedule: K network evaluations, no retraining.

Recap of Previous Lecture

The condition $\mathbf{y}$ can be a class label, a text prompt, or reference images.

Guidance modifies sampling to strengthen the influence of $\mathbf{y}$.

Classifier-Guided Estimate

For now, restrict $\mathbf{y}$ to **class labels**.

Train a noisy-data classifier $p(\mathbf{y}|\mathbf{x}_t) \approx q(\mathbf{y}|\mathbf{x}_t)$ at every noise level, providing t as an input.

$$\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) := \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t) \approx \nabla_{\mathbf{x}_t} \log q(\mathbf{x}_t|\mathbf{y}).$$

Guidance Scale

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) := \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \nabla_{\mathbf{x}_t} \log p(\mathbf{y}|\mathbf{x}_t).$$

γ controls guidance strength: $\gamma = 0$ gives the unconditional estimate; $\gamma = 1$ gives the conditional estimate.

Larger γ favors samples with higher class probability; very large values can reduce diversity and introduce artifacts.

Dhariwal P., Nichol A. Diffusion Models Beat GANs on Image Synthesis, 2021
Sadat S. et al. Eliminating Oversaturation and Artifacts of High Guidance Scales in Diffusion Models, 2025

Recap of Previous Lecture

Classifier-Free Guidance

$$\mathbf{s}^{\gamma}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y}) = (1 - \gamma) \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) + \gamma \cdot \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \mathbf{y})$$

CFG

1. Introduce the "absence of conditioning" label $\mathbf{y} = \varnothing$.
2. Identify the unconditional score model $\mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t) = \mathbf{s}_{\boldsymbol{\theta},t}(\mathbf{x}_t, \varnothing)$.
3. Train one model on paired data, replacing $\mathbf{y}$ with $\varnothing$ with fixed probability.
4. Evaluate the model twice per sampling step: with $\mathbf{y}$ and with $\varnothing$.

Ho J., Salimans T. Classifier-Free Diffusion Guidance, 2022
Holderrieth P., Erives E. An Introduction to Flow Matching and Diffusion Models, 2025

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

03 SDE Basics

Outline

01 **Continuous-Time Normalizing Flows (CNF)**

02 Continuity Equation for CNF Log-Likelihood

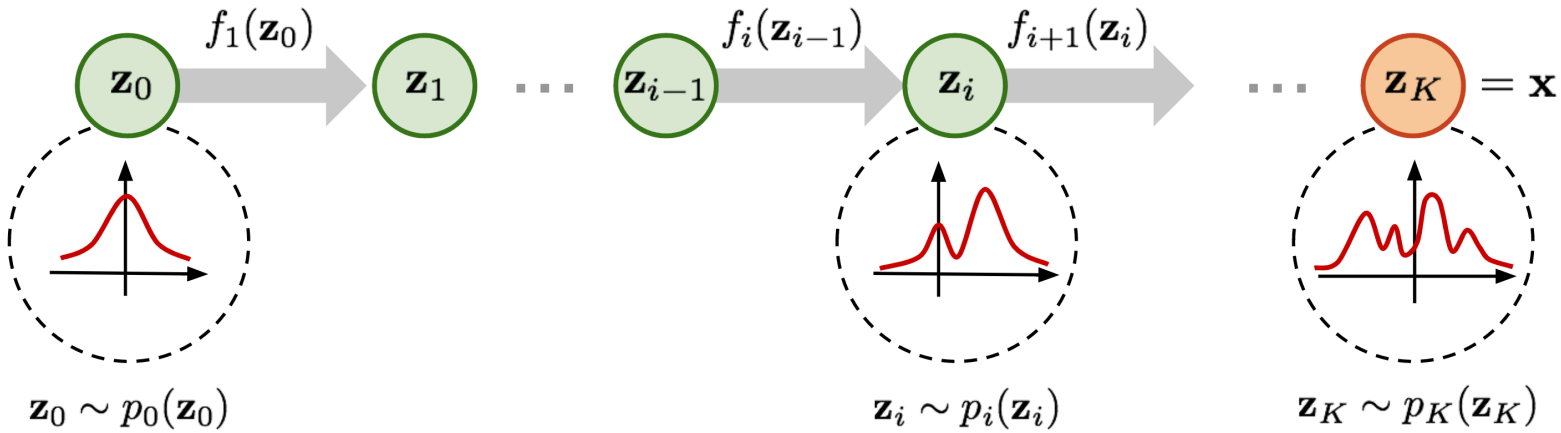
03 SDE Basics

Discrete-Time Normalizing Flows

Change of Variable Theorem (CoV)

Let $\mathbf{x}$ be a random variable with density $p(\mathbf{x})$, and let $\mathbf{f} : \mathbb{R}^m \to \mathbb{R}^m$ be a differentiable and **invertible** transformation. If $\mathbf{z} = \mathbf{f}(\mathbf{x})$, $\mathbf{x} = \mathbf{f}^{-1}(\mathbf{z}) = \mathbf{g}(\mathbf{z})$, then

$$p(\mathbf{x}) = p(\mathbf{z})|\det(\mathbf{J}_\mathbf{f})| = p(\mathbf{z})\left|\det\left(\frac{\partial \mathbf{z}}{\partial \mathbf{x}}\right)\right| = p(\mathbf{f}(\mathbf{x}))\left|\det\left(\frac{\partial \mathbf{f}(\mathbf{x})}{\partial \mathbf{x}}\right)\right|$$



$$\log p_{\boldsymbol{\theta}}(\mathbf{x}) = \log p(\mathbf{f}_K \circ \cdots \circ \mathbf{f}_1(\mathbf{x})) + \sum_{k=1}^{K} \log \left|\det\left(\frac{\partial \mathbf{f}_k}{\partial \mathbf{f}_{k-1}}\right)\right|.$$

Towards Continuous-Time Normalizing Flows

Up to this point, we have considered discrete-time normalizing flows:

$$\mathbf{x}_{t+1} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t, t); \quad \log p(\mathbf{x}_{t+1}) = \log p(\mathbf{x}_t) - \log \left| \det \frac{\partial \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t)}{\partial \mathbf{x}_t} \right|.$$

Residual Flows

Here $\mathbf{f}_{\boldsymbol{\theta}}$ is the NF **bijection**, $\mathbf{v}_{\boldsymbol{\theta}}$ is the **velocity** (vector field):

$$\mathbf{x}_{t+1} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}_t, t) = \mathbf{x}_t + \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_t, t)$$

Given $\mathbf{y} = \mathbf{f}_{\boldsymbol{\theta}}(\mathbf{x}, t)$, recover $\mathbf{x}$ by **fixed-point iteration**:

$$\mathbf{x} = \mathbf{y} - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}, t), \qquad \mathbf{x}^{(k+1)} = \mathbf{y} - \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}^{(k)}, t).$$

If $\mathbf{v}_{\boldsymbol{\theta}}$ has Lipschitz constant $L < 1$ in $\mathbf{x}$, the iteration is contractive. **Banach's theorem** guarantees a unique solution for every $\mathbf{y}$, so $\mathbf{f}_{\boldsymbol{\theta}}$ is invertible.

Behrmann J. et al. Invertible Residual Networks, 2019
Chen R. T. Q. et al. Residual Flows for Invertible Generative Modeling, 2019

Towards Continuous-Time Normalizing Flows

A residual step $\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$ is invertible if $hL < 1$:

$$\frac{\mathbf{x}(t+h) - \mathbf{x}(t)}{h} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Taking the limit $h \to 0$, we obtain continuous-time dynamics.

Continuous-Time Dynamics

Consider an Ordinary Differential Equation (ODE):

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t); \quad \text{with initial condition } \mathbf{x}(t_0) = \mathbf{x}_0.$$

$$\mathbf{x}(t_1) = \int_{t_0}^{t_1} \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\,dt + \mathbf{x}_0$$

The **velocity field** $\mathbf{v}_{\boldsymbol{\theta}} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ need not be contractive or invertible. Under suitable regularity conditions, **uniqueness of ODE solutions guarantees an invertible flow**.

ODEs, Flows and Numerical Solutions

Ordinary Differential Equation

$$\frac{d\mathbf{x}(t)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t), \qquad \mathbf{x}(t_0) = \mathbf{x}_0.$$

Flow

The **flow** $\boldsymbol{\psi} : \mathbb{R}^m \times [t_0, t_1] \to \mathbb{R}^m$ maps the initial point to its position at time t: $\mathbf{x}(t) = \boldsymbol{\psi}_t(\mathbf{x}_0)$.

$$\frac{d\boldsymbol{\psi}_t(\mathbf{x}_0)}{dt} = \mathbf{v}_{\boldsymbol{\theta}}(\boldsymbol{\psi}_t(\mathbf{x}_0), t), \qquad \boldsymbol{\psi}_{t_0}(\mathbf{x}_0) = \mathbf{x}_0.$$

Numerical Solution

$$\boldsymbol{\psi}_t(\mathbf{x}_0) = \mathbf{x}_0 + \int_{t_0}^{t} \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(s), s)\, ds \approx \texttt{ODESolve}_v(\mathbf{x}_0, \boldsymbol{\theta}, t_0, t).$$

A numerical solver approximates the flow by a sequence of **finite steps**.

Numerical Solution of ODEs: Euler

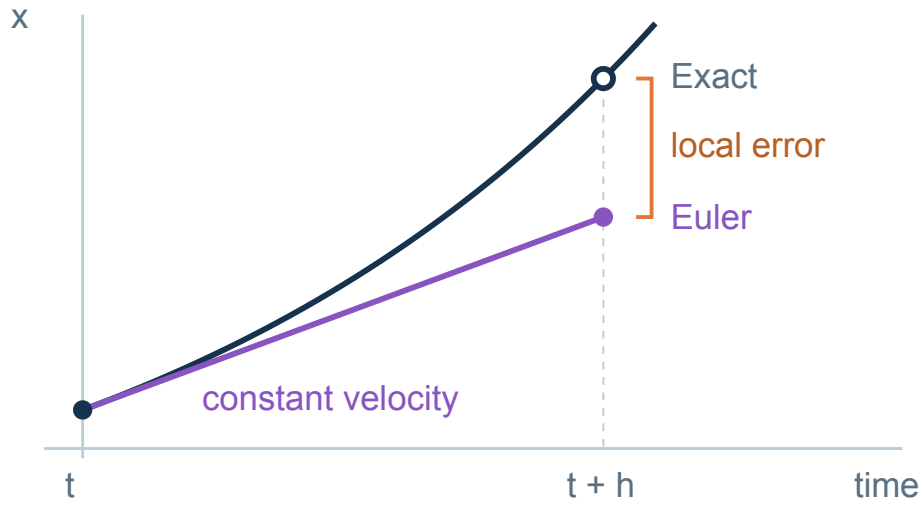
Euler Update Step

$$\frac{\mathbf{x}(t+h) - \mathbf{x}(t)}{h} = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h \cdot \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)$$

Freeze the current velocity over the step: **one field evaluation**.

One step: follow the tangent



One-Step Error

For a smooth field, Taylor expansion from the **exact starting point** gives

$$\mathbf{x}(t+h) = \mathbf{x}(t) + h\,\mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t) + \mathcal{O}(h^2).$$

Euler drops the higher-order terms: the **local error is** $\mathcal{O}(h^2)$.

Euler method; original one-step illustration
Driscoll T. A., Braun R. J. Fundamentals of Numerical Computation: Runge–Kutta methods

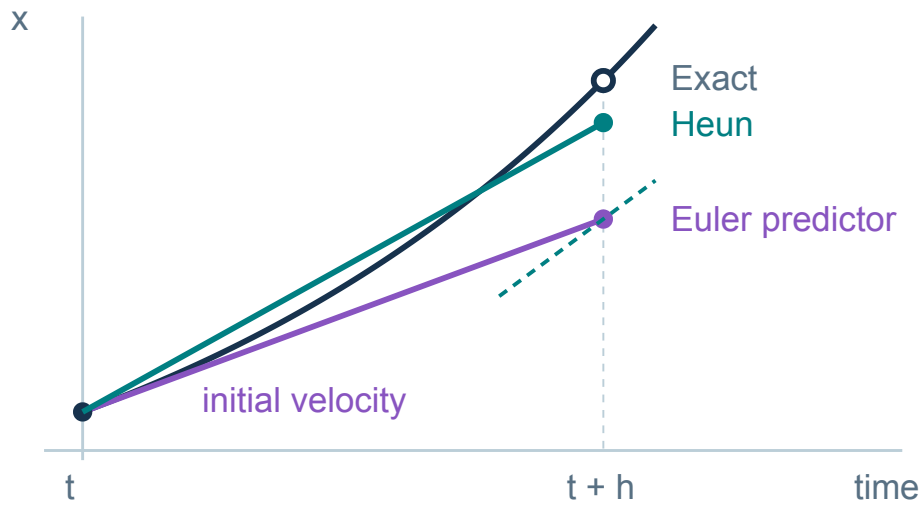
Numerical Solution of ODEs: Heun

Predict the endpoint

Evaluate the velocity at the start and take an Euler step of size h.

$$\mathbf{v}_0 = \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}_n, t_n), \qquad \widetilde{\mathbf{x}}_{n+1} = \mathbf{x}_n + h\,\mathbf{v}_0.$$

One step of Heun's method



Correct the step

Evaluate the velocity at the predicted endpoint. Advance from $\mathbf{x}_n$ using the **average of the two velocities**.

$$\mathbf{v}_1 = \mathbf{v}_{\boldsymbol{\theta}}(\widetilde{\mathbf{x}}_{n+1}, t_n + h), \qquad \mathbf{x}_{n+1} = \mathbf{x}_n + \frac{h}{2}(\mathbf{v}_0 + \mathbf{v}_1).$$

ODE Solvers: Accuracy and Cost

8 steps on [0, 3] · h = 0.375

For a smooth field on a fixed interval $[0, T]$, the **global error** after N steps of size $h = T/N$ is

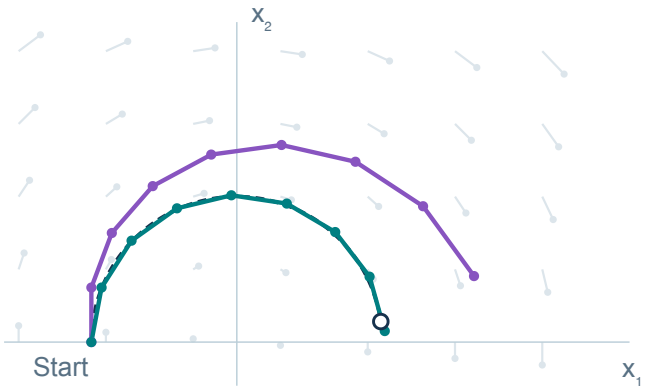
$$\|\widehat{\mathbf{x}}_N - \mathbf{x}(T)\| = \mathcal{O}(h^p).$$

Method	Global error	NFE per step
Euler	$\mathcal{O}(h)$	1
Heun	$\mathcal{O}(h^2)$	2
RK4	$\mathcal{O}(h^4)$	4

NFE counts calls to the velocity field. Halving h doubles the NFE needed to reach T.

$$\mathbf{v}(\mathbf{x}) = (x_2, -x_1), \quad \mathbf{x}(0) = (-1, 0)$$

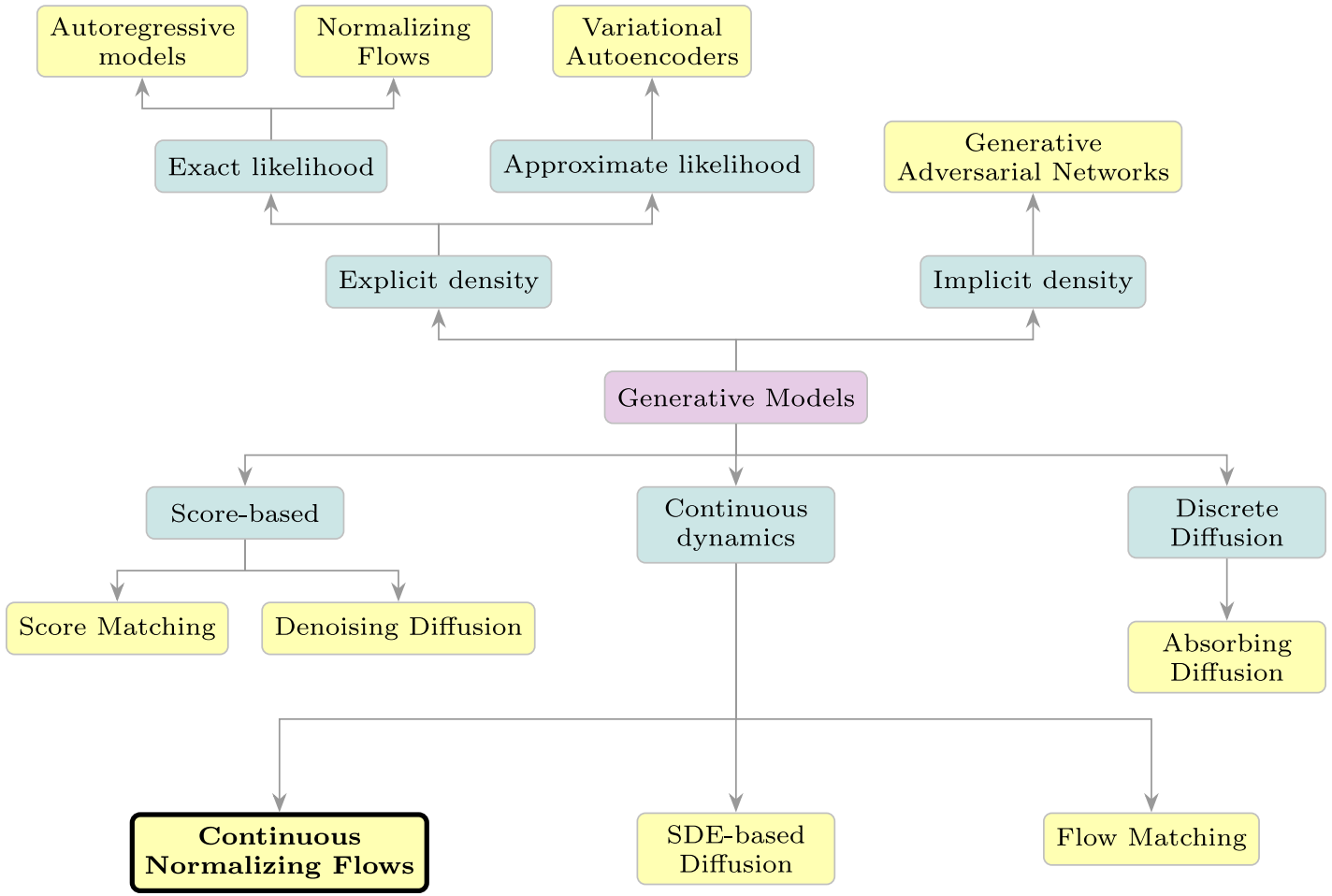
--- Exact — Euler — Heun



Step 8 / 8 · t = 3.000

Method	Distance to exact point	Total NFE
Euler	0.713	8
Heun	0.0707	16

Generative Models Taxonomy

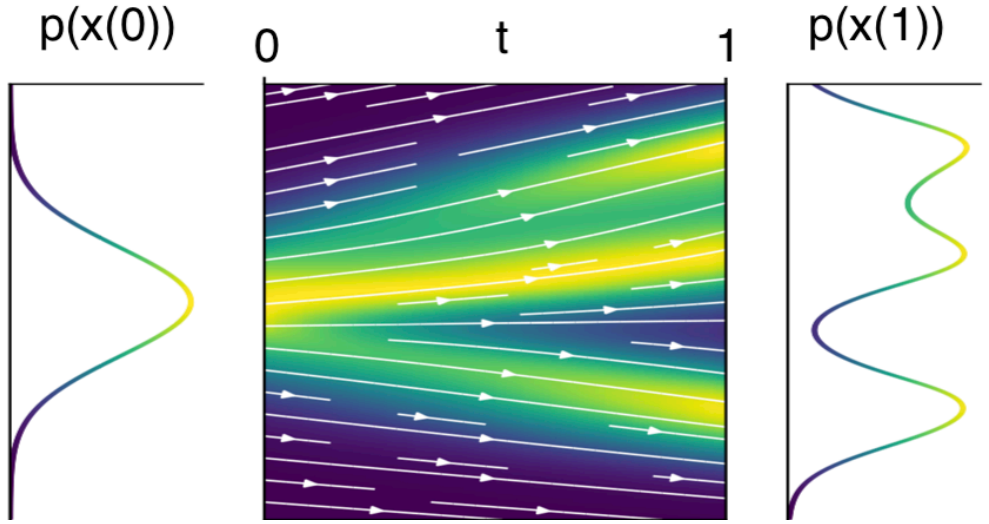


From ODE Trajectories to a Probability Path

Sampling the initial point selects an entire trajectory of the neural ODE.

$$\mathbf{x}(0) \sim p_0, \qquad \mathbf{x}(t) = \boldsymbol{\psi}_t(\mathbf{x}(0)), \qquad 0 \leq t \leq 1.$$

- Suppose $\mathbf{x}(0)$ is a random variable with density $p_0(\mathbf{x})$.
- Then, $\mathbf{x}(t)$ is a random variable with density $p_t(\mathbf{x})$.
- $p_t(\mathbf{x}) = p(\mathbf{x}, t)$ describes the **probability path** between $p_0(\mathbf{x})$ and $p_1(\mathbf{x})$.



Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

Continuous-Time Normalizing Flows: Reversibility

Theorem (Picard)

If $\mathbf{v}$ is continuously differentiable with a bounded derivative in $\mathbf{x}$ and continuous in t, then the ODE has a **unique solution** given by a flow $\boldsymbol{\psi}_t$.

This guarantees the ODE is **uniquely reversible**.

$$\boldsymbol{\psi}_1(\mathbf{x}_0) = \mathbf{x}_0 + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\boldsymbol{\psi}_t(\mathbf{x}_0), t)\, dt$$

$$\mathbf{x}(1) = \mathbf{x}(0) + \int_0^1 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\, dt$$

$$\mathbf{x}(0) = \mathbf{x}(1) + \int_1^0 \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)\, dt$$

Note: Unlike discrete-time flows, $\mathbf{v}$ need not be invertible (uniqueness ensures bijection).

To approximate the inverse flow with **Euler or Heun**, start at $\mathbf{x}(1)$ and take steps with $h < 0$ toward $t = 0$. The recovered point has discretization error.

> How can we compute $p_t(\mathbf{x})$ at arbitrary t?

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

03 SDE Basics

Continuous-Time NF

Theorem (Continuity Equation)

If $\mathbf{v}$ is continuously differentiable in $\mathbf{x}$ (with bounded derivative) and continuous in t, and $p_t(\mathbf{x}) > 0$, then

$$\frac{d \log p_t(\mathbf{x}(t))}{dt} = -\operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right)$$

Integrating gives the density **along the trajectory**.

Solution

$$\log p_1(\mathbf{x}(1)) = \log p_0(\mathbf{x}(0)) - \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

Hutchinson's trace estimator:

$$\operatorname{tr}(\mathbf{J}) = \mathbb{E}_{\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})}[\boldsymbol{\epsilon}^T \mathbf{J} \boldsymbol{\epsilon}], \qquad \mathbf{J} = \frac{\partial \mathbf{v}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}.$$

- One noise vector gives an **unbiased trace estimate** via autodiff, without forming the full Jacobian.
- **Exact identity; approximate computation** using an ODE solver and a stochastic trace estimate.

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

CNF: Training and Sampling

Fix $p_0 = \mathcal{N}(0, \mathbf{I})$ and learn $\mathbf{v}_{\boldsymbol{\theta}}$; the endpoint density is $p_1 = p_{\boldsymbol{\theta}}$.

Training

1. Sample $\mathbf{x} \sim p_{\text{data}}$. Solve the ODE **backward**, from $\mathbf{x}(1) = \mathbf{x}$ to $\mathbf{x}(0)$.
2. Compute the negative log-likelihood along this trajectory:

$$\mathcal{L} = -\log p_{\boldsymbol{\theta}}(\mathbf{x}) = -\log p_0(\mathbf{x}(0)) + \int_0^1 \operatorname{tr}\left(\frac{\partial \mathbf{v}_{\boldsymbol{\theta}}(\mathbf{x}(t), t)}{\partial \mathbf{x}(t)}\right) dt.$$

3. Update $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathcal{L}$; differentiate through the ODE, e.g., via the **adjoint method**.

Sampling

1. Sample $\mathbf{x}(0) \sim p_0$.
2. Solve the ODE **forward** and return the endpoint:

$$\mathbf{x}(1) = \texttt{ODESolve}_v(\mathbf{x}(0), \boldsymbol{\theta}, t_0 = 0, t_1 = 1) \sim p_{\boldsymbol{\theta}}.$$

Chen R. T. Q. et al. Neural Ordinary Differential Equations, 2018
Grathwohl W. et al. FFJORD: Free-form Continuous Dynamics for Scalable Reversible Generative Models, 2018

Outline

01 Continuous-Time Normalizing Flows (CNF)

02 Continuity Equation for CNF Log-Likelihood

03 SDE Basics

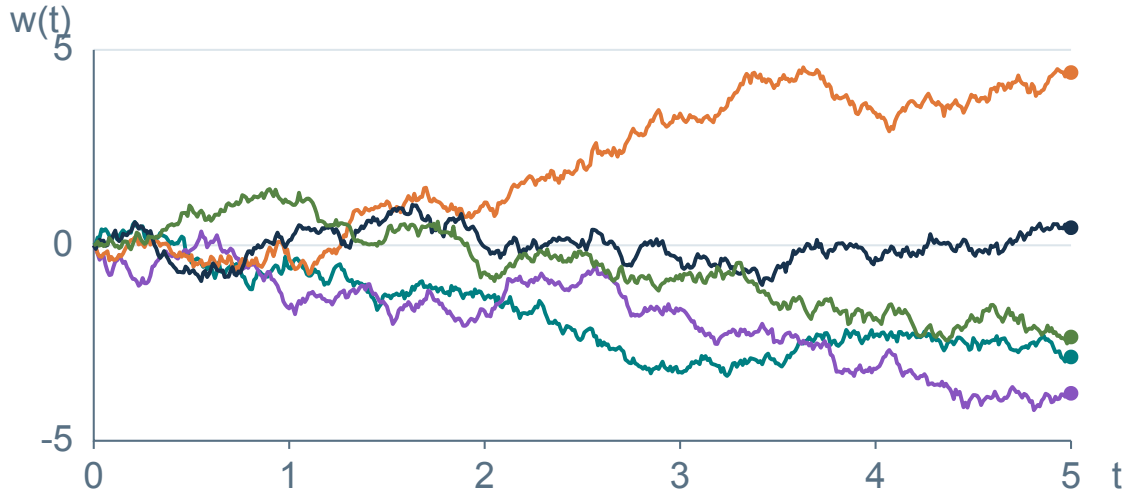
Stochastic Differential Equation (SDE)

In a CNF, fixing the initial point fixes the entire trajectory. What if noise also acts throughout the motion?

Wiener Process

$\mathbf{w}(t)$ is the standard Wiener process (Brownian motion), defined by:

1. $\mathbf{w}(0) = 0$ (almost surely);
2. $\mathbf{w}(t)$ has independent increments. For $s < t < u$,
   $\big(\mathbf{w}(t) - \mathbf{w}(s)\big) \perp \big(\mathbf{w}(u) - \mathbf{w}(t)\big)$.
3. $\mathbf{w}(t)$ trajectories are continuous;
4. $\mathbf{w}(t) - \mathbf{w}(s) \sim \mathcal{N}(0, (t-s)\mathbf{I})$ for $t > s$;



$$\Delta\mathbf{w} = \mathbf{w}(t + \Delta t) - \mathbf{w}(t) \sim \mathcal{N}(0, \Delta t\, \mathbf{I}), \qquad \Delta t > 0.$$

Here $d\mathbf{w}$ is **informal shorthand** for a small Brownian increment, modeled as $\sqrt{dt}\,\boldsymbol{\epsilon}$ with $\boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I})$.

Stochastic Differential Equation (SDE)

An SDE generalizes the dynamics of a CNF by adding Brownian noise to the motion induced by the velocity field.

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}, \qquad \mathbf{x}(0) \sim p_0.$$

- $\mathbf{f} : \mathbb{R}^m \times [0, 1] \to \mathbb{R}^m$ is the **drift** vector field.
- $g : [0, 1] \to \mathbb{R}$ is the **diffusion** coefficient. It scales the noise from the standard Wiener process $\mathbf{w}(t)$.

> Setting $g(t) = 0$ and $\mathbf{f} = \mathbf{v}_{\boldsymbol{\theta}}$ recovers the CNF dynamics.

Itô Integration Basics

The SDE should be understood in its **integral form** (Itô sense):

$$\mathbf{x}(t) = \mathbf{x}(0) + \int_0^t \mathbf{f}(\mathbf{x}(s), s)\, ds + \int_0^t g(s)\, d\mathbf{w}(s)$$

- The first integral is a classical (Riemann) integral.
- The second integral is an **Itô stochastic integral**.

$$\int_0^t g(s)\, d\mathbf{w}(s) = \lim_{N\to\infty} \sum_{i=0}^{N-1} g(t_i)\big(\mathbf{w}(t_{i+1}) - \mathbf{w}(t_i)\big)$$

- Brownian paths are continuous but a.s. **nowhere differentiable** $\Rightarrow$ Fundamental Theorem of Calculus fails.
- **Itô's lemma** is the stochastic counterpart of the chain rule; we will not need its explicit form here.
- Expressions $d\mathbf{x}$, dt, $d\mathbf{w}$ are **formal shorthand** for infinitesimal increments.

What Determines an SDE Trajectory?

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}$$

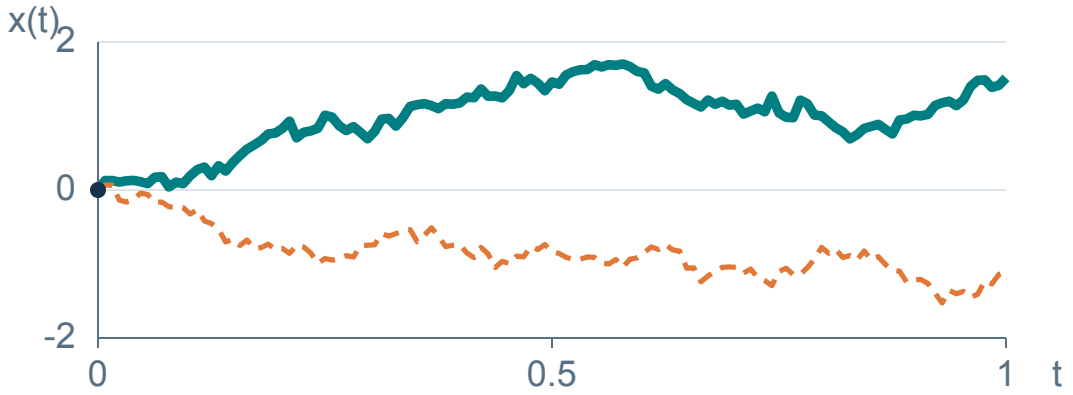
Existence and Uniqueness

If $\mathbf{f}$ is continuous in t and continuously differentiable in $\mathbf{x}$ with bounded derivative, and g is continuous, then the SDE has a unique strong solution for a given initial condition and Wiener process.

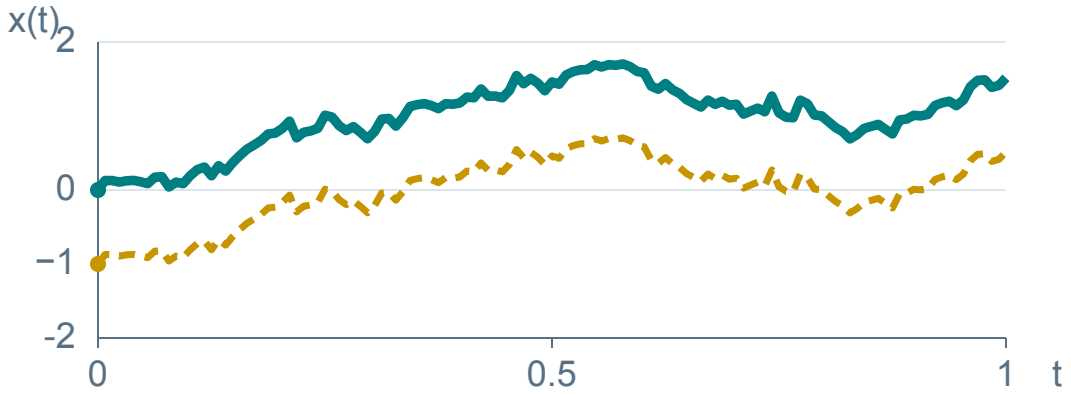
Two Sources of Randomness

For $dx = \frac{1}{2}\, dt + dw$, we can vary the initial draw $x_0 \sim p_0$ or the Brownian path $w(t)$.

Different Brownian paths, same $x_0 = 0$



Same Brownian path, $x_0 = 0$ and $x_0 = -1$



Fixing both the **initial point** and the **Brownian path** determines the trajectory almost surely.

Stochastic Differential Equation (SDE)

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}$$

Discretizing the SDE (Euler-Maruyama Update) – `SDESolve`

$$\mathbf{x}(t + dt) = \mathbf{x}(t) + \mathbf{f}(\mathbf{x}(t), t) \cdot dt + g(t) \cdot \boldsymbol{\epsilon} \cdot \sqrt{dt}$$

If $dt = 1$, then

$$\mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{f}(\mathbf{x}_t, t) + g(t) \cdot \boldsymbol{\epsilon}$$

- At any time t, the process has density $p_t(\mathbf{x}) = p(\mathbf{x}, t)$.
- $p : \mathbb{R}^m \times [0, 1] \to \mathbb{R}_+$ specifies a **probability path** from $p_0(\mathbf{x})$ to $p_1(\mathbf{x})$.

> How can we obtain the probability path $p_t(\mathbf{x})$ for $\mathbf{x}(t)$?

Stochastic Differential Equation (SDE)

$$d\mathbf{x} = \mathbf{f}(\mathbf{x}, t)\, dt + g(t)\, d\mathbf{w}, \quad d\mathbf{w} = \boldsymbol{\epsilon} \cdot \sqrt{dt}, \quad \boldsymbol{\epsilon} \sim \mathcal{N}(0, \mathbf{I}).$$

Theorem (Kolmogorov-Fokker-Planck)

If $p_t(\mathbf{x}) \in C^{1,2}$ (i.e., C^1 in t and C^2 in $\mathbf{x}$), then

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{f}(\mathbf{x}, t) p_t(\mathbf{x})\right) + \frac{1}{2} g^2(t) \Delta_{\mathbf{x}} p_t(\mathbf{x})$$

Divergence

$$\operatorname{div}(\mathbf{v}) = \sum_{i=1}^{m} \frac{\partial v_i(\mathbf{x})}{\partial x_i} = \operatorname{tr}\left(\frac{\partial \mathbf{v}(\mathbf{x})}{\partial \mathbf{x}}\right)$$

Laplacian

$$\Delta_{\mathbf{x}} p_t(\mathbf{x}) = \sum_{i=1}^{m} \frac{\partial^2 p_t(\mathbf{x})}{\partial x_i^2} = \operatorname{tr}\left(\frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right)$$

Equivalent trace form of the Fokker–Planck equation

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = \operatorname{tr}\left(-\frac{\partial}{\partial \mathbf{x}}\left[\mathbf{f}(\mathbf{x}, t) p_t(\mathbf{x})\right] + \frac{1}{2} g^2(t) \frac{\partial^2 p_t(\mathbf{x})}{\partial \mathbf{x}^2}\right)$$

From the Fokker–Planck Equation to Probability Flux

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x})\right) + \frac{1}{2}g^2(t)\Delta_{\mathbf{x}} p_t(\mathbf{x})$$

Combine drift and diffusion into one flux

Since $\Delta_{\mathbf{x}} p_t = \operatorname{div}(\nabla_{\mathbf{x}} p_t)$ and $g(t)$ is independent of $\mathbf{x}$,

$$\frac{\partial p_t(\mathbf{x})}{\partial t} = -\operatorname{div}\left(\underbrace{\mathbf{f}(\mathbf{x}, t)p_t(\mathbf{x}) - \tfrac{1}{2}g^2(t)\nabla_{\mathbf{x}} p_t(\mathbf{x})}_{\text{probability flux}}\right).$$

With $g(t) = 0$, only the drift flux remains. This gives the continuity equation for a CNF.

When is the density stationary?

We want $p_t(\mathbf{x}) = p_0(\mathbf{x})$ for every t. At each $\mathbf{x}$, the density must have zero time derivative:

$$0 = -\operatorname{div}\left(\mathbf{f}(\mathbf{x}, t)p_0(\mathbf{x}) - \tfrac{1}{2}g^2(t)\nabla_{\mathbf{x}} p_0(\mathbf{x})\right).$$

A Langevin Drift That Preserves the Target Density

For a smooth target density $p_0(\mathbf{x}) > 0$, stationarity requires

$$0 = -\operatorname{div}\big(\mathbf{f}(\mathbf{x}, t)p_0(\mathbf{x}) - \tfrac{1}{2}g^2(t)\nabla_{\mathbf{x}} p_0(\mathbf{x})\big)\,.$$

Choose a drift that makes the flux zero

One way to get zero divergence is to cancel the drift and diffusion fluxes at every $\mathbf{x}$:

$$\mathbf{f}(\mathbf{x}, t)p_0(\mathbf{x}) = \tfrac{1}{2}g^2(t)\nabla_{\mathbf{x}} p_0(\mathbf{x}).$$

Divide by the target density to obtain the score

$$\mathbf{f}(\mathbf{x}, t) = \tfrac{1}{2}g^2(t)\frac{\nabla_{\mathbf{x}} p_0(\mathbf{x})}{p_0(\mathbf{x})} = \tfrac{1}{2}g^2(t)\nabla_{\mathbf{x}} \log p_0(\mathbf{x}).$$

With $g(t) = 1$, the Langevin drift is half the score

$$d\mathbf{x} = \tfrac{1}{2}\nabla_{\mathbf{x}} \log p_0(\mathbf{x})\, dt + d\mathbf{w}.$$

If $\mathbf{x}(0) \sim p_0$, then $\mathbf{x}(t) \sim p_0$ for every t.

Langevin SDE (Special Case)

$$d\mathbf{x} = \tfrac{1}{2}\nabla_{\mathbf{x}} \log p_0(\mathbf{x})\, dt + d\mathbf{w}.$$

If $\mathbf{x}(0) \sim p_0$, then $\mathbf{x}(t) \sim p_0$ for every t.

Discretized Langevin SDE

For a step of length η, Euler–Maruyama gives

$$\mathbf{x}_{k+1} - \mathbf{x}_k = \frac{\eta}{2}\nabla_{\mathbf{x}_k} \log p_0(\mathbf{x}_k) + \sqrt{\eta}\,\boldsymbol{\epsilon}_k, \qquad \boldsymbol{\epsilon}_k \sim \mathcal{N}(0, \mathbf{I}).$$

Set the target density to the model density, $p_0(\mathbf{x}) = p_{\boldsymbol{\theta}}(\mathbf{x})$.

Langevin Dynamics

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \frac{\eta}{2}\nabla_{\mathbf{x}_k} \log p_{\boldsymbol{\theta}}(\mathbf{x}_k) + \sqrt{\eta}\,\boldsymbol{\epsilon}_k, \qquad \boldsymbol{\epsilon}_k \sim \mathcal{N}(0, \mathbf{I}).$$

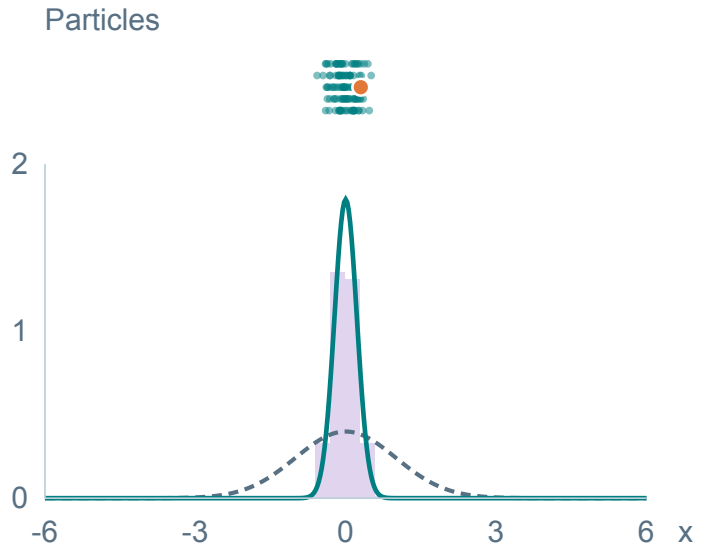
For a finite step size, this update approximates the continuous-time Langevin SDE.

Holderrieth P., Erives E. An Introduction to Flow Matching and Diffusion Models, 2025, §3.3
Welling M. Bayesian Learning via Stochastic Gradient Langevin Dynamics, 2011

Langevin: Moving Particles, Stationary Density

$$p_0(x) = p_*(x) = \mathcal{N}(0, 1) \qquad t = 3$$

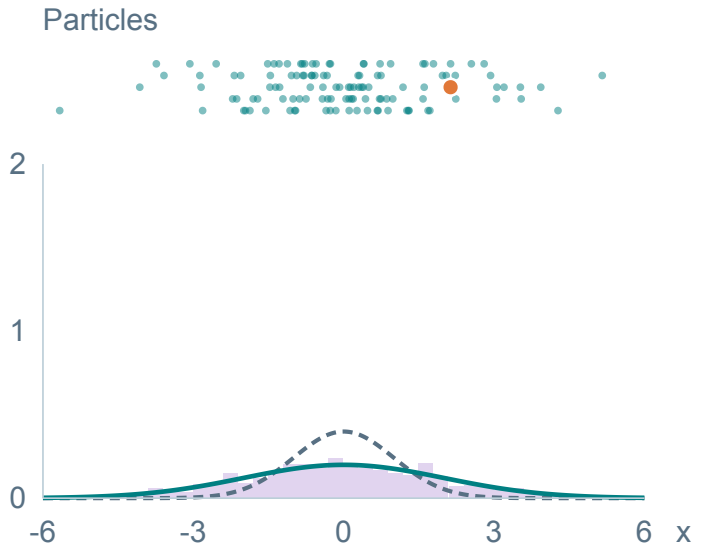
Drift only · $dx = -\frac{1}{2}x\,dt$



Variance 0.05

Drift pulls particles toward zero, narrowing the density.

Diffusion only · $dx = dw$



Variance 4.00

Random steps spread the particles out, widening the density.

Both · $dx = -\frac{1}{2}x\,dt + dw$



Variance 1.00

Drift balances diffusion, so particles move while the density stays fixed.

Histogram — Exact density --- Target density

Summary

- CNFs define invertible transformations via neural ODEs; likelihood-based training requires ODE solves, trace evaluations and gradients through the solver.
- If $\mathbf{x}_0$ is a random variable, this yields a **probability path** $p_t(\mathbf{x})$ as time evolves. The continuity equation describes the evolution of $\log p(\mathbf{x}, t)$ over time.
- An SDE defines a stochastic process with drift and diffusion terms; ODEs are a special case of SDEs.
- The KFP equation describes the probability dynamics of an SDE.
- With unit diffusion, a drift of half the target score makes the probability flux zero and keeps the target density stationary.